Findr

Software Requirements Specification — Campus Lost & Found Board

Document Version	2.0
Group	Group 7
Institution	Zeal College of Engineering & Research, Narhe, Pune
Project Type	College Internship Project
Project Duration	15 Days
Date	July 2026

1. Introduction

1.1 Purpose

This document specifies the software requirements for Findr, a web application that allows students to report, browse, claim, and resolve lost or found items within their college campus. It is intended for use by the developers, project guide, and evaluators as a reference for the system's intended functionality and scope.

1.2 Scope

The system allows a student to register and log in using their college-issued ZPRN ID and a password, report an item as lost or found, browse and filter all reported items on a shared board, submit a claim on an item with a message and contact info, and allow the original reporter to approve or reject claims, or mark an item resolved directly. A separate admin role provides platform-wide oversight, including removing inappropriate listings and viewing usage statistics. The system is built as a project-scale web application intended for demonstration and internal college use, not for large-scale production deployment.

1.3 Intended Audience

College students as end users, a designated admin (e.g., a faculty coordinator or student council member) for moderation, project evaluators/guides reviewing the submission, and the developers for future reference and extension of the project.

1.4 Definitions and Abbreviations

Term	Definition
ZPRN ID	Unique student identification number issued by the college, used as the login identifier
SRS	Software Requirements Specification
API	Application Programming Interface
JWT	JSON Web Token, used for session authentication
CRUD	Create, Read, Update, Delete — basic data operations

1.5 Technology Stack

Layer	Technology
Backend / Web Server	Node.js with Express.js
Database	MySQL (managed locally via XAMPP / phpMyAdmin)
Authentication	bcrypt (password hashing) + JSON Web Tokens
Frontend	HTML, CSS, vanilla JavaScript

2. Overall Description

2.1 Product Perspective

Findr is a standalone web application consisting of a REST API backend and a browser-based frontend. It is not integrated with any existing college ERP system; ZPRN ID is used only as a unique identifier field within this system's own database.

2.2 Product Functions

- Student account registration and login using ZPRN ID and password
- Reporting an item as lost or found, with description, category, location, and contact info
- Browsing, searching, and filtering all reported items on a shared board
- Submitting a claim with a message and contact info on an item believed to belong to the claimant
- Reviewing and approving/rejecting claims made on items the user has reported
- Marking a reported item as resolved directly, without requiring a claim
- Admin oversight: removing listings and viewing platform-wide statistics

2.3 User Characteristics

End users are college students with basic familiarity with web applications and no special technical training required to use the system. The admin role is expected to be held by a small number of trusted individuals responsible for moderating the platform.

2.4 Constraints

- Built within a 15-day project timeline as a college internship deliverable
- Uses MySQL as the database, managed locally via XAMPP, requiring Apache and MySQL services to be running in the development environment
- No integration with official college identity verification for ZPRN ID at present
- Admin accounts are assigned directly in the database; there is no self-service admin signup, by design

2.5 Assumptions and Dependencies

It is assumed that each student's ZPRN ID is unique and known only to that student. The system depends on Node.js, the associated npm packages (Express, mysql2, bcrypt, jsonwebtoken), and a running MySQL server being installed in the deployment environment.

3. Functional Requirements

ID	Requirement	Description
FR-1	User Registration	System shall allow a new user to sign up using name, ZPRN ID, and password. ZPRN ID must be unique.
FR-2	User Login	System shall authenticate a user via ZPRN ID and password, and issue a session token (JWT) on success.
FR-3	Report Item	An authenticated user shall be able to report an item with name, status (lost/found), category, location, contact info, and description.
FR-4	Browse & Filter Board	Any visitor shall be able to view all reported items, and filter by status, category, or location, or search by keyword.
FR-5	Submit Claim	An authenticated user shall be able to submit a claim with an optional message and contact info on any unclaimed item.
FR-6	View Claims	The system shall display all claims submitted on a given item, including the date each claim was made.
FR-7	Approve/Reject Claim	Only the original reporter of an item shall be able to approve or reject a claim on that item.
FR-8	Mark Item Resolved	The original reporter shall be able to mark their item as resolved directly, without needing an approved claim.
FR-9	Auto-update Item Status	When a claim is approved, or an item is marked resolved, the item's status shall automatically update to 'claimed'.
FR-10	Admin: Remove Listing	A user with the admin role shall be able to remove any item listing from the platform.
FR-11	Admin: View Statistics	A user with the admin role shall be able to view platform-wide counts of items, claims, and resolved items.
FR-12	Category Selection	When reporting an item, the user shall select a category from a predefined list rather than typing free text.
FR-13	Admin: View Action Log	A user with the admin role shall be able to view a log of every listing removal, including which admin performed it and when.
FR-14	User Dashboard Data	The system shall provide a user their own reported items and activity counts via queries scoped to their user ID, independent of their display name.

3.1 User Dashboard

Accessible to any registered, logged-in student. Shows the student's own activity on the platform, retrieved via dedicated database queries scoped to the logged-in user's ID (rather than filtering the full item list on the frontend), so results remain accurate even if two students share the same name.

Feature	Description
Dashboard Summary	Counts of items reported, claims made, and pending claims awaiting the student's review
My Reported Items	List of items the student has reported, with current status (lost/found/claimed)
My Claims	List of claims the student has submitted, with status (pending/approved/rejected)
Claim Approvals	Claims received on the student's own reported items, with approve/reject actions
Mark as Resolved	Directly close out a reported item without waiting on a claim
Report New Item	Shortcut to report a new lost or found item

3.2 Admin Dashboard

Accessible only to users with the 'admin' role (e.g., a faculty coordinator or student council member responsible for moderating the platform). Provides oversight across all users and items, rather than just the admin's own activity.

Feature	Description
All Items Overview	View every item reported on the platform, regardless of reporter
Remove Listings	Remove item listings that are inappropriate, duplicate, or resolved outside the app
Action Log	View a chronological record of every admin removal action, including who performed it and when
Platform Statistics	Summary counts — total items, total claims, and resolved items

4. External Interface Requirements

4.1 User Interface

The frontend is a single-page browser interface with five sections: Sign In/Sign Up, Report Item, The Board (browse, search, and filter items), My Items (view and resolve claims, mark items resolved), and Admin (visible in practice only to admin users, for listing moderation and platform statistics).

4.2 Hardware Interfaces

No special hardware is required. The system runs on any standard PC capable of running Node.js, a MySQL server, and a modern web browser.

4.3 Software Interfaces

Backend: Node.js runtime with Express framework. Database: MySQL, accessed via the mysql2 library. Frontend communicates with the backend over HTTP using JSON, via the Fetch API.

4.4 Communication Interfaces

The frontend and backend communicate over HTTP on localhost (port 3000 by default) during development. Authenticated requests include a bearer token in the Authorization header.

5. Non-Functional Requirements

Category	Requirement
Security	Passwords are stored as bcrypt hashes, never in plain text. Session access is controlled via signed JWT tokens. Admin-only routes are protected by a role check on the server.
Usability	The interface shall be simple enough for a first-time user to sign up and report an item without external instructions.
Performance	The system shall return item listings, including filtered/searched results, within 1 second under normal demo-scale load (fewer than 100 concurrent users).
Reliability	Core data (users, items, claims) shall persist across server restarts via the MySQL database.
Maintainability	Code is organized into separate files for database schema (db.js), server logic (server.js), and frontend (index.html) for ease of modification.
Portability	The application can run on any OS supporting Node.js and MySQL (Windows, macOS, Linux) with no code changes.

6. System Data Model

6.1 Database Overview

The Findr database consists of 5 tables, storing all user, item, claim, category, and admin activity data for the platform. The database engine is MySQL, managed locally via XAMPP/phpMyAdmin during development.

Table	Purpose	Fields	Primary Key	Foreign Keys
users	Registered student and admin accounts	6	id	None
items	Every lost/found item reported	10	id	reportedBy → users.id
claims	Claims submitted against items	7	id	itemId → items.id, claimantId → users.id
categories	Fixed list of item categories, used to populate the category dropdown	2	id	None
admin_logs	Record of every admin moderation action, for accountability	6	id	adminId → users.id

Note: items.category stores the category name directly rather than a strict foreign key to the categories table. This was a deliberate simplification to avoid rewiring the existing search and filter logic within the project timeline; the categories table is used to populate the category dropdown and keep entries consistent, without full relational enforcement. Full normalization (items.categoryId → categories.id) is listed under Future Scope.

6.2 Entities and Relationships

The system uses three related tables: Users, Items, and Claims. A user can report many items (one-to-many). An item can receive many claims (one-to-many). Each claim belongs to exactly one item and one claimant. A user's role field determines whether they have admin privileges.

6.3 Users Table

Field	Type	Notes
id	INT	Primary key, auto-increment
name	VARCHAR(100)	Required
zprnId	VARCHAR(50)	Unique, required — used for login
password	VARCHAR(255)	Stored as a bcrypt hash
role	VARCHAR(20)	'student' or 'admin' — defaults to 'student'
createdAt	DATETIME	Defaults to current timestamp

6.4 Items Table

Field	Type	Notes
id	INT	Primary key, auto-increment
itemName	VARCHAR(150)	Required
description	TEXT	Optional
category	VARCHAR(50)	Optional
location	VARCHAR(150)	Optional
imageUrl	VARCHAR(255)	Optional — path or link to a photo of the item
contactInfo	VARCHAR(150)	Contact details for the reporter (e.g. phone or email)
status	VARCHAR(20)	'lost', 'found', or 'claimed' — defaults to 'lost'
reportedBy	INT	Foreign key → users.id
dateReported	DATETIME	Defaults to current timestamp

6.5 Claims Table

Field	Type	Notes
id	INT	Primary key, auto-increment
itemId	INT	Foreign key → items.id
claimantId	INT	Foreign key → users.id
claimMessage	TEXT	Optional message from claimant
contactInfo	VARCHAR(150)	Contact details for the claimant (e.g. phone or email)
status	VARCHAR(20)	'pending', 'approved', or 'rejected' — defaults to 'pending'
dateClaimed	DATETIME	Date and time the claim was submitted — defaults to current timestamp

6.6 Categories Table

Field	Type	Notes
id	INT	Primary key, auto-increment
name	VARCHAR(50)	Unique — e.g. 'Bag', 'Electronics', 'ID Card'

6.7 Admin Logs Table

Field	Type	Notes
id	INT	Primary key, auto-increment
adminId	INT	Foreign key → users.id — the admin who performed the action
action	VARCHAR(100)	Description of the action taken, e.g. 'remove_item'
itemId	INT	The item the action was performed on, if applicable
itemName	VARCHAR(150)	Name of the item at the time of the action, kept for readability after deletion
timestamp	DATETIME	Defaults to current timestamp

7. Future Scope / Known Limitations

- Persist login tokens (e.g., browser storage) so sessions survive a page refresh
- Add rate-limiting on login to prevent brute-force attempts
- Integrate with official college systems for ZPRN ID verification
- Proper image upload flow (currently a manually entered link/path only)
- Real-time notifications when a claim is submitted or resolved
- Automatic archiving of old unclaimed items after a set period
- Full normalization of item categories (items.categoryId as a foreign key to categories.id, instead of a stored name)
- Introduce a reward points system to encourage students to report and return found items, with points redeemable for small campus incentives